Roteiro — demonstração do backend (Sprint 3), direto na aplicação

Sem slides: a apresentação é feita no app rodando. Duração sugerida: 8 a 10 minutos.

Antes de começar (checklist, 17h30)

Documentos reais já testados (guarde em uma pasta à mão): `modelo_politica_seguranca_informacao.pdf` (Governo Digital, nota 4,0), PoSIC UFPE (3,9) e PoSIC IFB (3,7). Não use a PoSIC do SLU-DF (numeração inconsistente).

- ☐ `docker compose ps` — `mongo` e `mongo-express` como *running*
- ☐ `.env` com `MONGO_URI` preenchida
- ☐ Índice FAISS presente: `ls data/processed/` (`normas.index`, `normas_trechos.json`, `normas_index.json`)
- ☐ Limpar os testes: `python db/limpar_testes.py` (simulação) e depois `python db/limpar_testes.py --apagar`
- ☐ Terminal 1 (raiz do repo), **sem `--reload`**: `uvicorn backend.main:app`
- ☐ Terminal 2: `streamlit run frontend/app.py`
- ☐ Abrir `http://localhost:8000/saude` → `api: ok`, `mongo: ok`, `busca_semantica: pronta` (aguarde ~30 s após subir a API)
- ☐ Abas abertas: app (8501), `/docs` (8000), mongo-express (8081)
- ☐ Arquivos prontos na pasta: `Politica_Seguranca_Nimbus_Logistica.pdf` e `..._Revisada.docx` (em `data/samples/`)
- ☐ Faça **uma auditoria de aquecimento** antes de apresentar (a primeira carrega o modelo)

Plano B: se a API cair, o app continua em modo simulado e avisa "Resultado SIMULADO". Diga isso com naturalidade: é o comportamento projetado. Se o Mongo cair, reinicie com `docker compose up -d` e suba a API de novo.

Sequência

1. **Contexto (30 s).** "O AutoAudit audita políticas de segurança da informação com PLN. Hoje mostro o backend da Sprint 3 funcionando de ponta a ponta."
2. **Status do sistema (30 s).** Sidebar do app e `/saude`: API, MongoDB e busca semântica no ar.
3. **Auditoria ao vivo (3 min).** Enviar a Nimbus (PDF). Mostrar: nota documental, lacunas por severidade, trecho do documento, problema, sugestão, referência (NIST/LGPD) e MITRE quando houver. Explicar uma lacuna alta (senhas vagas ou incidentes sem prazo/responsável/ANPD) e as "Normas próximas (busca semântica)".
4. **Correção e nova nota (1 min).** Enviar a Nimbus **revisada** (escrita seguindo as sugestões): nota 5,0, sem lacunas. Mensagem: o relatório é acionável.
5. **Documento real (1 min, opcional).** Enviar o modelo de política do Governo Digital (nota 4,0, 9 lacunas médias/baixas) ou a PoSIC da UFPE/IFB. Mostrar que o motor lê PDF de 12 a 23 páginas, detecta capítulos e acha lacunas verificáveis. Avisar: são políticas de órgãos públicas, em estilo normativo; a nota é indicativa.
6. **Persistência (1 min).** Aba Histórico do app (lida do MongoDB) e mongo-express (coleções `auditorias`, `relatorios`, `normas`).
7. **API (1 min).** `/docs`: `POST /auditorias`, `GET /relatorios`. O frontend só consome o contrato.
8. **Resiliência (30 s, opcional).** Mostrar o modo simulado desligando a API só se houver tempo e confiança.
9. **Limites e próximos passos (1 min).** Ver abaixo.

O que dizer sobre como funciona

- **Pipeline:** arquivo → texto (pypdf / python-docx) → seções e sentenças → regras de detecção → lacunas → nota → busca semântica de normas → grava no MongoDB.
- **Por que regras (regex) e não LLM:** resultado determinístico, explicável e auditável; cada lacuna aponta a frase e a regra. Cabe a um trabalho de PLN com base em padrões linguísticos (termos vagos, ausência de prazo/responsável).
- **Nota documental:** pesos alta = 3, média = 2, baixa = 1; `5 × (1 − peso_total / (3 × máx(nº de seções, 5)))`, mínimo 0. Saúde de Compliance = nota documental enquanto a Verificação Prática não roda; depois, média das duas.
- **FAISS:** índice de 1.705 trechos de NIST SP 800-53 e LGPD com embeddings `paraphrase-multilingual-mpnet-base-v2`; liga a lacuna às normas mais próximas, com limiar por família para filtrar ruído.
- **MongoDB:** coleções `normas`, `auditorias` (ciclo pendente → processando → concluída/erro) e `relatorios`, com regras de integridade no `db/repositorio.py`.

- **Validação:** gabarito da Nimbus com 6 lacunas plantadas: 6/6 detectadas, 0 falsos positivos; política revisada: 0 lacunas.

Perguntas prováveis

- **"Funciona com qualquer documento?"** PDF com texto e DOCX até 10 MB, em português. PDF escaneado (imagem) não, pois não há OCR; o sistema avisa em vez de analisar errado.
- **"E se o documento for bom?"** A nota sobe e as lacunas somem (exemplo da versão revisada). Mas as regras cobrem um conjunto limitado de problemas (senha, MFA, incidente, legislação genérica, dispositivo pessoal, DPO, termos vagos); nota alta não é certificação.
- **"Por que não um LLM?"** Ver acima; fica como evolução, usando o FAISS como base de recuperação.
- **"E a Verificação Prática?"** Não foi executada nesta versão: `nota_pratica` e `verificacoes` voltam vazios e o frontend trata esse estado. É a próxima etapa (pentest controlado em laboratório).
- **"Privacidade dos documentos?"** O processamento é local; nada é enviado a serviços externos. O banco guarda texto das lacunas e o hash do arquivo.
- **"Como a nota é calibrada?"** Heurística provisória, ajustada nas políticas de teste; calibrar com mais documentos reais é próximo passo.

Limitações assumidas (diga você antes de perguntarem)

- Documentos de órgãos públicos (PoSIC) têm estilo normativo (capítulos, incisos); o motor reconhece "CAPÍTULO", títulos numerados e em maiúsculas, mas PDFs com numeração inconsistente geram seções imprecisas (ex.: PoSIC do SLU-DF).
- Títulos sem numeração só são reconhecidos se estiverem em maiúsculas ou no estilo Título do Word.
- Regras em português apenas.
- Só uma regra de ausência (Encarregado/DPO).
- Regra de senha aceita qualquer valor numérico mencionado, mesmo fraco.
- Verificação Prática ainda não implementada.

Testar um documento de empresa antes (30 s)

```
python backend/analisar_arquivo.py caminho/do/documento.pdf
```

Mostra as seções detectadas, a nota e as lacunas sem subir API nem banco. Se vier "Preâmbulo" ou "Documento" com tudo dentro, o documento não tem títulos reconhecíveis e a nota é pouco confiável.